



Guide de Portabilité - JobbingTrack

Ce guide documente toutes les améliorations apportées pour rendre le projet **JobbingTrack** complètement portable et installable sur n'importe quel système supportant Docker.



Objectifs de Portabilité

Le projet JobbingTrack est maintenant **100% portable** et peut être installé et utilisé sur :

- **Linux** (Ubuntu, Debian, CentOS, Fedora, Arch Linux, etc.)
- **macOS** (avec Docker Desktop)
- **Windows** (avec WSL ou Docker Desktop)
- **Tout système** supportant Docker et Docker Compose



Améliorations Apportées

1. Détection Automatique Docker Compose

Problème résolu : Les scripts et Makefiles utilisaient directement `docker-compose` sans vérifier si l'utilisateur avait la version standalone ou le plugin Docker Compose.

Solution implémentée :

- Création d'un **wrapper portable** : `scripts/utils/docker-compose-wrapper.sh`
- Détection automatique de la commande disponible
- Support transparent pour `docker-compose` et `docker compose`

```
# Le wrapper détecte automatiquement :
if command -v docker-compose &> /dev/null; then
    echo "docker-compose" # Version standalone
elif docker compose version &> /dev/null 2>&1; then
    echo "docker compose" # Plugin Docker v2+
else
    echo "Erreur: Docker Compose non disponible"
fi
```

2. Scripts Shell Portables

Problèmes résolus :

- Shebangs non portables (`#!/bin/bash` au lieu de `#!/usr/bin/env bash`)
- Utilisation de `pgrep` (non disponible sur macOS)

- Chemins absolus problématiques

Solutions implémentées :

- **Shebangs portables** : `#!/usr/bin/env bash` dans tous les scripts
- Remplacement de `pgrep` par `ps aux | grep` (plus portable)
- Utilisation de chemins relatifs

3. Makefiles Portables

Problèmes résolus :

- Utilisation directe de `docker-compose` dans les Makefiles
- Conditions redondantes `if command -v docker-compose`
- Utilisation de `netstat` (non disponible sur tous les systèmes)

Solutions implémentées :

- **Fonctions wrapper** dans `makefiles/shared/common.mk`
- Détection automatique et fonctions portables
- Remplacement de `netstat` par `ss / netstat` avec fallback

```
# Fonction portable pour vérifier les ports
define check_port_occupied
    @if command -v ss &> /dev/null; then \
        ss -tuln | grep -q ":%(1) "; \
    elif command -v netstat &> /dev/null; then \
        netstat -tuln 2>/dev/null | grep -q ":%(1) "; \
    else \
        echo "⚠ Impossible de vérifier les ports"; \
        exit 1; \
    fi
endef
```

4. Fonctions Système Portables

Nouvelles fonctions dans `makefiles/shared/common.mk` :

- `check_docker` : Vérification portable de Docker
- `docker_compose` : Wrapper pour Docker Compose
- `check_port_occupied` : Vérification portable des ports
- `get_port_pid` : Récupération portable du PID d'un port

Structure des Fichiers Modifiés

Scripts Shell

```
scripts/
├─ utils/docker-compose-wrapper.sh    #  Wrapper Docker Compose portable
├─ core/check.sh                      #  Mis à jour avec wrapper
├─ core/start.sh                      #  Mis à jour avec wrapper
├─ core/stop.sh                      #  Mis à jour (pgrep → ps aux)
├─ db/seed.sh                        #  Mis à jour avec wrapper
├─ db/backup.sh                      #  Mis à jour avec wrapper
├─ setup/install-dependencies.sh      #  Déjà portable (multi-distrib)
└─ health/check-all.sh               #  Mis à jour avec wrapper
```

Makefiles

```
makefiles/
├─ shared/common.mk                  #  Fonctions portables ajoutées
├─ Makefile.production              #  Mis à jour avec wrapper
├─ Makefile                         #  Mis à jour avec wrapper
└─ root/Makefile                    #  Mis à jour avec wrapper
```



Tests de Portabilité

Un script de test complet a été créé : `test_portability.sh`

```
# Exécuter les tests de portabilité
./test_portability.sh

# Résultat attendu :
 Docker Compose (standalone)
 Docker
 ss (outil ports moderne)
 scripts/core/check.sh
 scripts/core/start.sh
 scripts/core/stop.sh
 scripts/db/seed.sh
 scripts/db/backup.sh
 scripts/utils/docker-compose-wrapper.sh
 Makefile
 makefiles/Makefile.production
 makefiles/Makefile
 makefiles/root/Makefile
```

Utilisation Portable

Démarrage du projet

```
# Ces commandes fonctionnent sur tous les systèmes :  
make up           # Démarrer les services essentiels  
make up-full      # Démarrer tous les services  
make logs         # Voir les logs  
make status       # Statut des services  
make down         # Arrêter tous les services
```

Scripts shell

```
# Ces scripts sont maintenant portables :  
./scripts/core/start.sh           # Démarrage complet  
./scripts/core/check.sh --detailed # Vérification système  
./scripts/db/seed.sh              # Seed de la base de données  
./scripts/db/backup.sh            # Sauvegarde
```

Commandes Système Supportées

Docker et Docker Compose

-  **Docker** : Détection automatique
-  **Docker Compose** : Support `docker-compose` et `docker compose`
-  **Docker Desktop** : Compatible (Windows/macOS)

Outils de vérification

-  **ss** : Outil moderne (Linux)
-  **netstat** : Outil classique (avec fallback)
-  **ps aux** : Alternative portable à `pgrep`

Gestionnaires de paquets

-  **apt-get** : Debian/Ubuntu
-  **yum** : RHEL/CentOS
-  **pacman** : Arch Linux
-  **brew** : macOS (via script d'installation)

Prérequis Système

Minimum

- **Docker** 20.10+
- **Docker Compose** 1.29+ (standalone) ou **Docker** 2.0+ (plugin)
- **Bash** 4.0+
- **make** 3.81+
- **curl** ou **wget**

Recommandé

- **ss** (outil de vérification de ports)
- **git** (pour le contrôle de version)

Dépannage

Si Docker Compose n'est pas détecté

```
# Vérifier la détection
make check-deps

# Installer Docker Compose si nécessaire
# Linux : sudo apt install docker-compose-plugin
# macOS/Windows : Utiliser Docker Desktop
```

Si des ports sont occupés

```
# Vérifier les ports
make status

# Nettoyer les conflits
make clean-conflicts
```

Si les scripts échouent

```
# Vérifier la syntaxe
bash -n scripts/core/start.sh
```

```
# Exécuter en mode debug  
bash -x scripts/core/start.sh
```

Conclusion

Le projet **JobbingTrack** est maintenant **complètement portable** et peut être :

1. **Installé** sur n'importe quel système supportant Docker
2. **Exécuté** sans modification du code
3. **Déployé** en production sur différents environnements
4. **Maintenu** sans se soucier des différences système

Toutes les commandes et scripts s'adaptent automatiquement à l'environnement d'exécution ! 